StockDate

Food Bank Inventory and Expiration Manager

Phase 1: Planning and Analysis (Weeks 1–5)

Team Members:

Hugo Cobo Diaz Guerra

Ángel Luis Segovia Medina

Francisco Ingelmo

Enrique Barrera Ruperez

Universidad Alfonso X el Sabio (UAX)

Academic Year 2026-2027

Contents

1 Introduction and State of the Art **3**
1.1 Problem Context . . . 3
1.1.1 System Environment . . . 3
1.1.2 Why is the current process inefficient? . . . 3
1.1.3 Who suffers from this problem? . . . 3
1.2 SMART Goals and Key Performance Indicators (KPIs) . . . 4
1.3 State of the Art Analysis . . . 5
1.3.1 Comparison Criteria . . . 5
1.3.2 Comparative Matrix . . . 6
1.3.3 Summary and Competitive Advantage . . . 8

2 Requirements Engineering **8**
2.1 Stakeholders Analysis and Primary Concerns . . . 8
2.1.1 Internal Stakeholders . . . 9
2.1.2 External Stakeholders . . . 9
2.1.3 Stakeholder Summary Matrix . . . 10
2.2 Functional Requirements and Use Case Specifications . . . 10
2.2.1 Overview of System Use Cases . . . 10
2.2.2 UC-01: User Authentication and Login . . . 11
2.2.3 UC-02: Record New Stock Batch Entry . . . 12
2.2.4 UC-03: Search and Filter Inventory Stock . . . 13
2.2.5 UC-04: Process Stock Dispatch/Checkout . . . 14
2.2.6 UC-05: Expiration Dashboard Status Monitoring . . . 15
2.2.7 UC-06: Process Discard/Spoiled Stock Write-Off . . . 16
2.2.8 UC-07: Generate Monthly Waste Summary Report . . . 17
2.2.9 UC-08: Manage User Roles and Accounts . . . 18
2.3 Non-Functional Requirements (NFR) and Testing Verification . . . 19
2.3.1 Performance Requirements . . . 19
2.3.2 Security Requirements . . . 19
2.3.3 Reliability Requirements . . . 20
2.3.4 Usability Requirements . . . 20
2.3.5 Maintainability Requirements . . . 21
2.3.6 Portability Requirements . . . 21
2.4 Categorized Non-Functional Requirements Summary . . . 22

3 Architectural Design **23**
3.1 Introduction and Pattern Selection . . . 23
3.2 Justification Based on Non-Functional Requirements . . . 23
3.3 High-Level Component Diagram . . . 24
3.4 Layer Decoupling Strategies . . . 24

3.5 Static View: UML Class Diagram 25
3.6 Core Classes and Data Types 27
3.7 Interfaces and Data Transfer Objects (DTOs) 27
3.8 Multiplicities and System Relationships 28
3.9 Applied Design Patterns 28
3.10 Dynamic View: Sequence Diagrams 29
3.10.1 Scenario 1: New Food Batch Registration Flow 29
3.10.2 Scenario 2: Dashboard Alert Retrieval Flow 30
3.11 Technology Decision Records (ADRs) 31
3.11.1 ADR-01: Selection of Backend Framework 31
3.11.2 ADR-02: Selection of Database Engine 32
3.11.3 ADR-03: Deployment Architecture 33
3.11.4 ADR-04: Web Rendering Strategy 34
3.12 Architectural Conclusions 35
3.13 Requirement-to-Component Traceability Matrix 35
3.14 Technical References 37

1. Introduction and State of the Art

1.1 Problem Context

1.1.1 System Environment

StockDate is designed for small non-profit organizations, local food banks, community pantries and small neighborhood stores. In these environments, product intake is highly unpredictable: centers suddenly receive large amounts of donated or discounted food without fixed schedules or standard packaging formats.

Unlike major commercial distribution warehouses, these entities depend heavily on volunteers and non-technical staff with frequent turnover. Furthermore, they do not have the financial budget for expensive software licenses or IT maintenance. Therefore, they need a simple, lightweight tool that requires no complex onboarding or training.

1.1.2 Why is the current process inefficient?

Currently, most of these centers manage food stock using paper notebooks or simple spreadsheets like Microsoft Excel or Google Sheets. This creates three major operational problems in their day-to-day workflow:

- **Lack of automated tracking and expiration alerts:** A spreadsheet only highlights food that is about to expire if someone sets up conditional formatting rules by hand and nothing warns the team unless they open the file and look. In practice, volunteers end up checking boxes and shelves one by one. As a result, older items get pushed to the back and eventually thrown away.
- **Frequent human errors and wasted time:** Entering data manually without proper input validation leads to typos, wrong dates, duplicate records or unrecorded exits. This causes inventory mismatches and forces staff to spend hours correcting errors.
- **Food waste and financial loss:** Throwing food away due to overlooked expiration dates is a serious issue. For an NGO, it means losing food that could have helped families in need; for small businesses, it results in direct financial loss and extra waste disposal fees.

1.1.3 Who suffers from this problem?

These inefficiencies directly affect three main groups:

1. **Food store/bank coordinators and directors:** Instead of spending time building partnerships, securing funding and expanding distribution, they spend hours manually reconciling stock numbers and fixing spreadsheet errors.
2. **Field staff and volunteers:** Volunteers who offer their time to help end up frustrated by tedious, repetitive tasks like counting cans one by one or filling out paper forms.

3. **Final beneficiaries and families in need:** When food is wasted due to poor expiration management, less variety and volume of food reach the people who truly need it.

1.2 SMART Goals and Key Performance Indicators (KPIs)

To ensure the project progresses steadily over the 16-week timeline, five SMART (Specific, Measurable, Achievable, Relevant, Time-bound) goals have been set. Each goal includes a Key Performance Indicator (KPI) and the conditions under which it will be measured. All KPIs are measured during Phase 3 (weeks 12–14).

- **Goal 1: Fast Batch Registration and Checkout**
  - **Description:** Make logging incoming and outgoing food batches as fast and simple as possible by minimizing required input fields.
  - **SMART Criteria:** Specific to stock logging, measured in seconds, achievable with a single short form, and evaluated before the end of the 16 weeks.
  - **KPI:** Average time to register a new food batch under 45 seconds. To have a fair baseline, the same test participants will also time the task in a spreadsheet, so both figures are measured under the same conditions.
- **Goal 2: Food Waste Reduction via Automated Expiration Alerts**
  - **Description:** Build a visual dashboard that sorts items by expiration date (FEFO: First Expired, First Out) and shows clear visual warnings for products nearing their expiration date.
  - **SMART Criteria:** Directly targets food waste, measured with a repeatable simulation, achievable with the dashboard and FEFO rules of UC-04 and UC-05.
  - **KPI:** In a simulation where the same set of 50 batches with mixed expiration dates is managed for four simulated weeks, once with a spreadsheet and once with StockDate, the number of units that expire without being dispatched is at least 30% lower with StockDate.
- **Goal 3: System Query Performance and Fast Response Time**
  - **Description:** Ensure that searching, filtering, and checking stock status give near-instant results on the web interface.
  - **SMART Criteria:** Measured directly through server response time with a load-testing tool.
  - **KPI:** 95th percentile (p95) latency of search and filter requests under 200 ms, with 10,000 batches in the database and 5 concurrent users.

- **Goal 4: Code Reliability and Automated Test Coverage**
  - **Description:** Verify that critical business logic such as stock arithmetic and expiration date calculation runs reliably without bugs.
  - **SMART Criteria:** Measured automatically with a code coverage tool (JaCoCo) on every build.
  - **KPI:** At least 80% line coverage from unit and integration tests on the core backend services (InventoryManager, ExpirationAlertService and ReportService).
- **Goal 5: Ease of Use for Non-Technical Staff/Volunteers**
  - **Description:** Design a clean, simple user interface so any volunteer can use the application without needing a complicated manual.
  - **SMART Criteria:** Evaluated through hands-on testing with non-technical test participants.
  - **KPI:** 90% of test participants can log a new product, check expiring stock and complete a dispatch in under 5 minutes after a 2-minute introduction.

1.3 State of the Art Analysis

To better understand where StockDate stands, we compared it against the two software approaches most commonly used by small non-profits and local retail centers:

1. **Traditional Spreadsheets (Microsoft Excel / Google Sheets):** This is the most common approach among small entities because it requires no initial setup. However, spreadsheets are designed for general calculations and static data tables, not for day-to-day tracking of perishable goods.
2. **Enterprise ERP Systems (Odoo / SAP):** These are large platforms built for managing companies, supply chains and accounting. Their inventory modules are powerful, but their setup cost and complexity make them impractical for local food banks.

1.3.1 Comparison Criteria

To evaluate each alternative against the real needs of a food bank, we used four criteria:

- **Features and Business Logic:** Whether the software provides dedicated tools for perishable inventory management, such as automated expiration alerts (FEFO rule), role-based access control (admin vs. volunteer views) and waste reporting.
- **Scalability and Performance:** How each system behaves as data volume grows or when several users access it at the same time, checking for stability, latency or file corruption risks.

- **Implementation and Upkeep Costs:** Both direct licensing fees and indirect expenses, such as hosting, user training and technical support.
- **Tech Stack and Architecture:** The underlying technology, installation complexity, hardware requirements and deployment options.

1.3.2 Comparative Matrix

Table 1: Comparative matrix: StockDate vs. spreadsheets vs. ERP systems

Criteria	**StockDate (Our System)**	**Competitor 1: Spreadsheets (Excel/Sheets)**	**Competitor 2: ERP Systems (Odoo/SAP)**
Features	• Quick CRUD operations for food batches. • Dashboard with color-coded expiration warnings, calculated automatically. • Full history of entries, dispatches and discards. • Monthly waste reports. • Role-Based Access Control (Admin and Volunteer).	• Basic data tables and lists. • Color alerts only if someone configures conditional formatting by hand, and only visible when the file is opened. • No input validation, roles or movement history; cells can be deleted by accident.	• Full enterprise suite for sales, accounting, and logistics, including lot and expiration tracking. • Far more than a small food bank needs. • Steep learning curve for volunteers.
Scalability	Medium: enough for a single food bank with a few simultaneous users. SQLite allows one writer at a time (see ADR-02); the repository interfaces make a later move to PostgreSQL possible.	Very Low: becomes slow with thousands of rows, files can get corrupted, and multi-user editing of desktop files is unreliable.	Very High: built for large companies with several warehouses, but needs significant server resources.

Criteria	**StockDate (Our System)**	**Competitor 1: Spreadsheets (Excel / Sheets)**	**Competitor 2: ERP Systems (Odoo / SAP)**
Cost	No licence cost: built with free open-source tools (Java, Spring Boot, SQLite) and runs on any computer with Java 17 or newer.	Free (Google Sheets) to low (per-user Microsoft 365 subscription).	High: Odoo Community is free to download, but setup, hosting, customization and support are expensive. Odoo Enterprise and SAP also charge per-user licences.
Tech Stack	• Frontend: HTML, CSS, JavaScript. • Backend: Java (Spring Boot). • Database: SQLite. • Deployment: executable JAR (Docker image optional, see ADR-03).	Desktop files (XLSX, optional VBA macros) or cloud web apps (Google Workspace).	Odoo: Python with PostgreSQL. SAP: mainly ABAP on its own database platform. Both need dedicated servers or paid cloud hosting.

1.3.3 Summary and Competitive Advantage

Spreadsheets cost little, but expiration control depends on someone setting up the rules and checking the file, and there is no validation, role control or history of movements. ERP systems do cover these needs, but they bring setup costs and a level of complexity that a volunteer-run food bank cannot manage.

StockDate sits between the two: a free tool focused only on stock and expiration management, simple enough for a new volunteer to use after a short introduction (Goal 5).

2. Requirements Engineering

This section details the software requirements specification for *StockDate*. Based on the problem context, target audience, and technology stack defined in Phase 1 (Java Spring Boot, SQLite, HTML/CSS/JS), the requirements focus strictly on food bank and small business inventory management without introducing out-of-scope features such as native mobile apps, physical hardware scanners, or external email/SMS notifications.

Note: Some things said below may change when we start advancing in the project, because we are constantly modifying the project and maybe we change something while programing.

For now, the section is organized into three parts:

1. **Stakeholder Analysis:** Identifies key actors and defines their primary concerns and operational demands.
2. **Use Case Specifications:** Provides complete functional specifications, explicitly identifying the **Trigger** that initiates each task and the core **Business Rules** governing its execution.
3. **Non-Functional Requirements:** Groups system quality attributes by category (Performance, Security, Reliability, Usability, Maintainability, Portability) and defines the **Testing Method** used to verify each requirement.

2.1 Stakeholders Analysis and Primary Concerns

StockDate will operate in environments such as local food banks, community pantries and small retail stores, where technical skills vary, volunteer turnover is high and resources are limited. Rather than presenting a simple list, this section evaluates each stakeholders role, main concerns and interface requirements.

The system has only two user roles, and these names are used throughout the rest of the document:

- **ADMIN:** the food bank coordinator who manages users, discards and reports.

- **VOLUNTEER:** field staff who register entries, search stock and log dispatches.

2.1.1 Internal Stakeholders

1. Food Bank Coordinator (ADMIN role)

- **Primary Concern: Security, Role Control and Operational Oversight.**
- **Detailed Analysis:** Coordinators manage staff permissions, oversee the inventory (stock levels and expiration dates) and set up user accounts. Their main concern is security and role isolation: volunteers must not be able to change system settings, delete records or modify user roles by accident. A strict Role-Based Access Control (RBAC) model solves this.

2. Volunteers and Field Staff (VOLUNTEER role)

- **Primary Concern: Speed, Interface Simplicity and Low Input Effort.**
- **Detailed Analysis:** Volunteers handle high-volume routine tasks: registering incoming food, searching for items and logging dispatches. Because shifts change constantly and technical skills vary, their main concern is speed and simplicity. They need an interface that allows fast logging, needs no complex training and has no multi-level menus.

2.1.2 External Stakeholders

1. Final Beneficiaries

- **Primary Concern: Food Safety, Quality and Service Consistency.**
- **Detailed Analysis:** Beneficiaries rely on the food bank for safe provisions. Their main concern is receiving food in good condition. The software must enforce First-Expired First-Out (FEFO) logic and block the dispatch of batches whose use-by date has passed.

2. External Health Inspectors and Compliance Auditors

- **Primary Concern: Traceability, Regulatory Compliance and Data Integrity.**
- **Detailed Analysis:** Health authorities require traceability for food distribution. Their main concern is verifying that entries, dispatches and discards of spoiled products are recorded with permanent timestamps and the user who performed them.

2.1.3 Stakeholder Summary Matrix

Table 2 maps each stakeholder to their primary concerns and core system requirements within *StockDate*.

Stakeholder	Primary Concern	Core System Requirement
Coordinator (ADMIN)	Security, Access Control & System Integrity	Role-based access control (RBAC), password hashing, protected admin routes.
Volunteer (VOLUNTEER)	Speed, Simplicity & Minimal Effort	Short forms, fast page loads, batch registration under 45 seconds.
Final Beneficiaries	Food Safety & Quality	FEFO sorting, visual warnings for expiring food, no dispatch of expired use-by batches.
Health Auditors	Traceability & Compliance	Permanent, timestamped movement records for entries, dispatches and discards.

Table 2: Stakeholder Analysis and System Requirement Mapping

2.2 Functional Requirements and Use Case Specifications

This subsection describes the functional requirements of *StockDate* as formal Use Case specifications. As required by the project guidelines, every specification details the "Trigger" that starts the workflow and the "Business Rules" that constrain it.

2.2.1 Overview of System Use Cases

- **UC-01:** User Authentication and Login
- **UC-02:** Record New Stock Batch Entry
- **UC-03:** Search and Filter Inventory Stock
- **UC-04:** Process Stock Dispatch/Checkout (FEFO Enforcement)
- **UC-05:** Expiration Dashboard Status Monitoring
- **UC-06:** Process Discard/Spoiled Stock Write-Off
- **UC-07:** Generate Monthly Waste Summary Report
- **UC-08:** Manage User Roles and Accounts

2.2.2 UC-01: User Authentication and Login

Use Case ID	**UC-01**
Use Case Name	User Authentication and Login
Primary Actor	All users (Admin, Volunteer)
Description	Authenticates user credentials and directs the user to the main view that matches their role.
Trigger	**The user opens the StockDate web application URL (login view).**
Pre-conditions	The Spring Boot server is running and a user account exists.
Post-conditions	The user has an authenticated HTTP session (session cookie) with role-specific permissions.
Main Flow	1. User navigates to the login screen. 2. User enters Username and Password. 3. User submits credentials. 4. System checks the password against the stored BCrypt hash. 5. System creates the user session and opens the main interface for that role.
Alternative Flows	**4a. Invalid Credentials:** System displays "Invalid username or password" and clears the password field. **4b. Deactivated Account:** System rejects the login and displays "This account is deactivated. Contact your coordinator."
Business Rule(s)	**1. User passwords must be at least 8 characters long and stored as BCrypt hashes.** **2. Inactive user sessions must automatically end after 45 minutes.**

Table 3: Use Case Specification: UC-01 User Authentication

2.2.3 UC-02: Record New Stock Batch Entry

Use Case ID	**UC-02**
Use Case Name	Record New Stock Batch Entry
Primary Actor	All users (Admin, Staff/Volunteer)
Description	Registers a new food donation or incoming batch, including product, category, quantity, storage location, expiration date and date type.
Trigger	**A volunteer receives a new delivery or donation of food items at the facility.**
Pre-conditions	User is logged in.
Post-conditions	A new batch record and its ENTRY movement are stored in SQLite, and the batch appears on the expiration dashboard.
Main Flow	1. User opens the "Add New Batch" form. 2. User enters Product Name, selects Category, and enters Quantity, Storage Location, Expiration Date and Date Type (use-by or best-before). 3. User submits the batch entry. 4. System validates the input fields. 5. System saves the batch and the ENTRY movement, and shows a success message to the user.
Alternative Flows	**4a. Missing Required Data:** System highlights empty mandatory fields in red and prevents submission. **4b. Use-by Date in the Past:** System blocks submission and displays "A use-by date cannot be in the past". **4c. Best-before Date in the Past:** System asks the user to confirm that the product is still in good condition before saving.
Business Rule(s)	**1. Quantities must be positive integers ($Quantity > 0$).** **2. Batches with a use-by date must have $ExpirationDate \geq CurrentDate$.** **3. Every new batch creates an ENTRY movement with timestamp and user.**

Table 4: Use Case Specification: UC-02 Record New Batch Entry

2.2.4 UC-03: Search and Filter Inventory Stock

Use Case ID	**UC-03**
Use Case Name	Search and Filter Inventory Stock
Primary Actor	All users (Admin, Staff/Volunteer)
Description	Allows users to search available food stock by product name, category or expiration status.
Trigger	**A user needs to check item availability or locate specific products in storage.**
Pre-conditions	User is logged in.
Post-conditions	A filtered table of matching stock items is displayed on screen.
Main Flow	1. User opens the Inventory Search view. 2. User enters a keyword or selects a category or status filter. 3. System runs the query against the SQLite database. 4. System shows matching active batches with Name, Category, Quantity, Location, Expiration Date and status.
Alternative Flows	**4a. No Matching Stock:** System displays "No inventory records match the search criteria."
Business Rule(s)	**1. Search must meet the latency target of NFR-PERF-01 (p95 $<$ 200 ms).** **2. Results are sorted by expiration date and show the same status labels as the dashboard (UC-05).**

Table 5: Use Case Specification: UC-03 Search and Filter Inventory Stock

2.2.5 UC-04: Process Stock Dispatch/Checkout

Use Case ID	**UC-04**
Use Case Name	Process Stock Dispatch/Checkout
Primary Actor	All users (Admin, Staff/Volunteer)
Description	Logs the removal of food items for distribution to beneficiaries while enforcing First-Expired First-Out (FEFO) logic.
Trigger	**A volunteer prepares food packages or items for distribution.**
Pre-conditions	User is logged in.
Post-conditions	Batch quantity is reduced, a DISPATCH movement is recorded, and batches that reach zero are marked inactive.
Main Flow	1. User searches for and selects the product to dispatch. 2. System shows the active batches of that product sorted by FEFO (earliest expiration first), with the first one preselected. 3. User confirms the batch, enters the dispatch quantity and submits. 4. System reduces the batch quantity and records the DISPATCH movement.
Alternative Flows	**2a. No Active Stock:** System shows "No stock available for this product." **2b. Expired Use-by Batch:** System shows the batch greyed out and it cannot be selected. **3a. Quantity Exceeds Stock:** System blocks the dispatch and shows the maximum available units. **3b. User Skips the Earliest Batch:** System asks for a mandatory override reason before continuing.
Business Rule(s)	**1. Stock can never drop below zero ($Quantity \geq 0$).** **2. Choosing a batch other than the earliest expiring one requires an override reason, which is stored in the movement.** **3. Batches whose use-by date has passed cannot be dispatched.**

Table 6: Use Case Specification: UC-04 Process Stock Dispatch

2.2.6 UC-05: Expiration Dashboard Status Monitoring

Use Case ID	**UC-05**
Use Case Name	Expiration Dashboard Status Monitoring
Primary Actor	All users (Admin, Staff/Volunteer)
Description	Provides a web control panel with color-coded warnings for items approaching expiration.
Trigger	**The user opens or refreshes the main dashboard view.**
Pre-conditions	User is logged in and the application is connected to the database.
Post-conditions	The dashboard shows current expiration alerts and summary counts.
Main Flow	1. User navigates to the main dashboard. 2. System calculates the days until expiration of each active batch using the server date. 3. System groups batches into tiers: Expired (< 0 days), Red (0–7 days), Yellow (8–14 days), Green (> 14 days). 4. System displays the count per tier and the list of pinned items.
Alternative Flows	**3a. Expired Stock Present:** System displays a banner at the top listing the expired batches that need a write-off (UC-06). **3b. Database Error:** System displays an error banner instead of the dashboard, never an empty "no alerts" view.
Business Rule(s)	**1. Days until expiration are always based on the current server date; the summary is rebuilt when stock changes or the date changes.** **2. Expired and Red items must stay pinned at the top of the dashboard.** **3. Every status color is shown together with a text label (e.g. "Expires in 3 days").**

Table 7: Use Case Specification: UC-05 Expiration Dashboard Monitoring

2.2.7 UC-06: Process Discard/Spoiled Stock Write-Off

Use Case ID	**UC-06**
Use Case Name	Process Discard/Spoiled Stock Write-Off
Primary Actor	Admin
Description	Registers damaged, spoiled, or expired food that must be removed from active stock, so that waste can be measured.
Trigger	**The dashboard flags expired items or a physical inspection finds damaged food.**
Pre-conditions	User is logged in with the ADMIN role.
Post-conditions	Batch quantity is reduced and a DISCARD movement is recorded with its reason.
Main Flow	1. Admin opens the Discard screen. 2. System lists expired batches first, followed by the rest of the active stock. 3. Admin selects the batch, enters the discarded quantity and selects a reason (Expired, Damaged, Spoiled, Other). 4. Admin confirms the write-off. 5. System updates the batch and records the DISCARD movement.
Alternative Flows	**3a. Partial Batch Discard:** Admin enters a quantity lower than the batch total; the remaining units stay active.
Business Rule(s)	**1. Discards can only be processed by users with the ADMIN role.** **2. Every write-off must include a discard reason.**

Table 8: Use Case Specification: UC-06 Process Discard Write-Off

2.2.8 UC-07: Generate Monthly Waste Summary Report

Use Case ID	**UC-07**
Use Case Name	Generate Monthly Waste Summary Report
Primary Actor	Admin
Description	Compiles monthly statistics on entries, dispatches and discards to support purchasing and donation decisions.
Trigger	**An admin selects a month and starts report generation.**
Pre-conditions	User is logged in with the ADMIN role.
Post-conditions	Summary metrics are shown on screen and can be exported as a CSV file.
Main Flow	1. Admin opens the Reports view. 2. Admin selects the month and year. 3. Admin clicks "Generate Report". 4. System reads the movement records of that month and computes total received, dispatched and discarded units. 5. System displays the totals, the waste rate and a chart. 6. Admin can export the report as a CSV file.
Alternative Flows	**4a. No Movements Found:** System displays "No transaction records found for the selected period." **4b. No Units Left Stock:** If nothing was dispatched or discarded, the waste rate is shown as "N/A".
Business Rule(s)	**1. Waste rate formula (units of the selected month):** Waste Rate (%) $= \left(\frac{\text{Discarded Units}}{\text{Dispatched Units+Discarded Units}}\right) \times 100$ **2. Report totals must match the movement records in SQLite exactly.**

Table 9: Use Case Specification: UC-07 Generate Monthly Waste Report

The waste rate uses the units that left the stock in the month (dispatched plus discarded) as the denominator. Using received units instead would mix food that arrived in one month with food discarded in another, and the rate could go above 100%.

2.2.9 UC-08: Manage User Roles and Accounts

Use Case ID	**UC-08**
Use Case Name	Manage User Roles and Accounts
Primary Actor	Admin
Description	Creates, edits, resets passwords for, or deactivates user accounts, and assigns roles (ADMIN or VOLUNTEER).
Trigger	**An admin needs to onboard a new volunteer or change someone's access.**
Pre-conditions	User is logged in with the ADMIN role.
Post-conditions	User data and role assignments are updated in the SQLite database.
Main Flow	1. Admin opens the User Management panel. 2. Admin selects "Create User" or edits an existing account. 3. Admin enters Username, Full Name and Role, and sets or resets the Password. 4. Admin saves the profile. 5. System validates the input, hashes the password and updates the user table.
Alternative Flows	**4a. Duplicate Username:** System displays "Username is already registered." **4b. Self-Deactivation Attempt:** System blocks an admin from deactivating their own account. **4c. Last Admin:** System blocks deactivating or changing the role of the last active admin.
Business Rule(s)	**1. At least one active ADMIN account must exist at all times.** **2. Plaintext passwords are never stored (BCrypt, which includes a salt).**

Table 10: Use Case Specification: UC-08 User Account Management

2.3 Non-Functional Requirements (NFR) and Testing Verification

Non-functional requirements set performance limits, quality standards and security constraints. So that each one can be checked objectively, every requirement is grouped by quality category and paired with its "Testing Verification Method".

2.3.1 Performance Requirements

NFR-PERF-01: Search Query Latency

- **Category:** Performance.
- **Requirement:** Stock search and filter requests shall have a 95th percentile (p95) response time under 200 ms, with 10,000 batches in the database and 5 concurrent users.
- **Testing Verification Method: Verified using automated load testing (Apache JMeter).** A seed script fills the database with 10,000 batches, and a JMeter plan sends search requests from 5 concurrent users for 5 minutes. The p95 value is read from the JMeter report.

NFR-PERF-02: Batch Registration Processing Time

- **Category:** Performance.
- **Requirement:** The backend shall process and save a new batch (request received to response sent) in under 1,000 ms.
- **Testing Verification Method: Verified using automated API tests.** A JMeter plan sends POST requests to the batch registration endpoint and measures the server response time, without including browser rendering.

2.3.2 Security Requirements

NFR-SEC-01: Password Hashing and Credential Security

- **Category:** Security.
- **Requirement:** Plaintext user passwords shall never be saved. Passwords must be hashed using BCrypt before being stored.
- **Testing Verification Method: Verified using direct database inspection and code review.** Inspecting the SQLite users table confirms that the password column only contains BCrypt hash strings (prefix "$2a$" or "$2b$").

NFR-SEC-02: Role-Based Route Protection

- **Category:** Security.
- **Requirement:** Admin endpoints (user management, discards and reports) must reject requests from VOLUNTEER sessions.
- **Testing Verification Method: Verified using automated integration tests.** Spring Boot integration tests log in as a volunteer and call every admin endpoint, checking that each one returns HTTP 403 Forbidden.

2.3.3 Reliability Requirements

NFR-REL-01: Application Availability

- **Category:** Reliability.
- **Requirement:** During an 8-hour continuous run (one working day), the application shall answer at least 99% of health checks successfully.
- **Testing Verification Method: Verified using a health-check monitoring script.** The application exposes the Spring Boot Actuator endpoint `/actuator/health`. A script calls it once per minute during the 8-hour run, logs the result, and calculates the percentage of successful responses.

NFR-REL-02: Database Transaction Integrity

- **Category:** Reliability.
- **Requirement:** SQLite write operations must be atomic: if the process stops during a write, the batch and its movement record are either both saved or both missing.
- **Testing Verification Method: Verified using process fault-injection testing.** The application process is forcibly terminated ("SIGKILL") during batch insertion. After restart, a check confirms that no batch exists without its ENTRY movement and that the database file passes SQLite's integrity check.

2.3.4 Usability Requirements

NFR-USA-01: Volunteer Onboarding and Task Efficiency

- **Category:** Usability.
- **Requirement:** Non-technical users must be able to register a product, check expiring stock and complete a dispatch in under 5 minutes after a 2-minute introduction.

- **Testing Verification Method: Verified through User Acceptance Testing (UAT) with non-technical participants.** Each session records task completion time and number of errors per participant.

NFR-USA-02: Visual Contrast and Color Accessibility

- **Category:** Usability.
- **Requirement:** Status badges and text must meet WCAG 2.1 level AA contrast (at least 4.5:1 for normal text). Status must never be shown by color alone; each badge also has a text label.
- **Testing Verification Method: Verified using automated accessibility audits.** Lighthouse or axe DevTools scans are run on the dashboard, search and dispatch screens, and no contrast errors may remain.

2.3.5 Maintainability Requirements

NFR-MAIN-01: Test Coverage of Core Logic

- **Category:** Maintainability.
- **Requirement:** Unit and integration tests shall cover at least 80% of the lines in InventoryManager, ExpirationAlertService and ReportService (Goal 4).
- **Testing Verification Method: Verified using JaCoCo coverage reports.** The Maven build generates the report on every run, and the percentage is checked before each delivery.

NFR-MAIN-02: Static Code Quality

- **Category:** Maintainability.
- **Requirement:** The main branch shall have no high-priority PMD violations and no Checkstyle errors.
- **Testing Verification Method: Verified using PMD and Checkstyle Maven plugins.** Both tools run as part of the build, and their reports are reviewed in Phase 3.

2.3.6 Portability Requirements

NFR-PORT-01: Cross-Platform Execution

- **Category:** Portability.
- **Requirement:** The complete application (backend, web assets and SQLite database file) shall run as a single executable JAR (`java -jar stockdate.jar`) on Windows,

macOS and Linux with Java 17 or newer, without installing a separate database server. A Docker image is an optional extension (see ADR-03).

- **Testing Verification Method: Verified using clean-machine deployment tests.** The JAR is started on a computer with each operating system that has only Java installed, and a short smoke test (login, register a batch, open the dashboard) is run.

2.4 Categorized Non-Functional Requirements Summary

Table 11 summarizes all Non-Functional Requirements by category, with their target and testing method:

Requirement ID	Category	Requirement Target	Test Verification Method
NFR-PERF-01	Performance	Search p95 $<$ 200 ms (10,000 batches, 5 users)	Apache JMeter load test.
NFR-PERF-02	Performance	Batch save on server $<$ 1,000 ms	JMeter API test on POST endpoint.
NFR-SEC-01	Security	BCrypt password hashing	SQLite table inspection and code review.
NFR-SEC-02	Security	Admin route protection	Integration tests (HTTP 403 checks).
NFR-REL-01	Reliability	$\geq$ 99% successful health checks in 8 h	Actuator health-check monitoring script.
NFR-REL-02	Reliability	Atomic writes	Fault-injection testing ("SIGKILL").
NFR-USA-01	Usability	Task completion $<$ 5 min	UAT with non-technical users.
NFR-USA-02	Usability	WCAG 2.1 AA contrast, text labels	Lighthouse / axe audits.
NFR-MAIN-01	Maintainability	$\geq$ 80% line coverage on core services	JaCoCo report.
NFR-MAIN-02	Maintainability	No high-priority PMD / Checkstyle issues	PMD and Checkstyle Maven plugins.
NFR-PORT-01	Portability	Executable JAR on Windows, macOS, Linux	Clean-machine smoke test.

Table 11: Non-Functional Requirements Categorization and Testing Summary

3. Architectural Design

3.1 Introduction and Pattern Selection

For StockDate, we selected a Layered Architecture. This pattern strictly separates the user interface from the database logic. Using Java with the Spring Boot framework naturally supports this structure through its built-in component model.

The system is divided into three main levels that communicate in a single, downward direction:

- **Presentation Layer (Frontend & Controllers):** The web interface is built with HTML, CSS, and JavaScript and runs in the browser. It calls Spring Boot REST controllers, which receive the HTTP requests, pass them to the logic layer and return JSON (see ADR-04).
- **Business Logic Layer (Services):** This is the core of StockDate. It contains all the inventory rules, such as FEFO (First Expired, First Out) sorting, batch validation, the expiration status calculation and the waste report.
- **Data Access Layer (Repositories):** This layer handles all reading and writing. It uses Spring Data JPA to translate Java objects into SQL queries against the SQLite database.

3.2 Justification Based on Non-Functional Requirements

This choice addresses the project's non-functional requirements and constraints:

- **Maintainability:** Layer separation isolates changes. For example, if a food bank wants the red alert to start at 10 days instead of 7, only the threshold used by ExpirationAlertService changes (it is read from `application.properties`). The frontend and the database schema stay the same. This separation also makes the core services easy to unit test (NFR-MAIN-01).
- **Ease of Deployment:** The compiled application, the web assets and the embedded SQLite database file are delivered as a single executable JAR. A food bank without IT staff only needs Java installed to run it (NFR-PORT-01). Packaging it as a Docker image later would not require any change to the layers.
- **Team Development Speed:** The four team members can work in parallel without blocking each other. Two members can build the web screens using mock JSON data, while the other two write the business logic and database queries. Since the layers communicate through agreed methods and DTOs, merge conflicts in GitHub are reduced.
- **Performance:** SQLite reads quickly but only allows one writer at a time. The business logic layer validates all input (for example, negative quantities or invalid dates) before

opening a database transaction, so invalid requests never hold the write lock.

3.3 High-Level Component Diagram

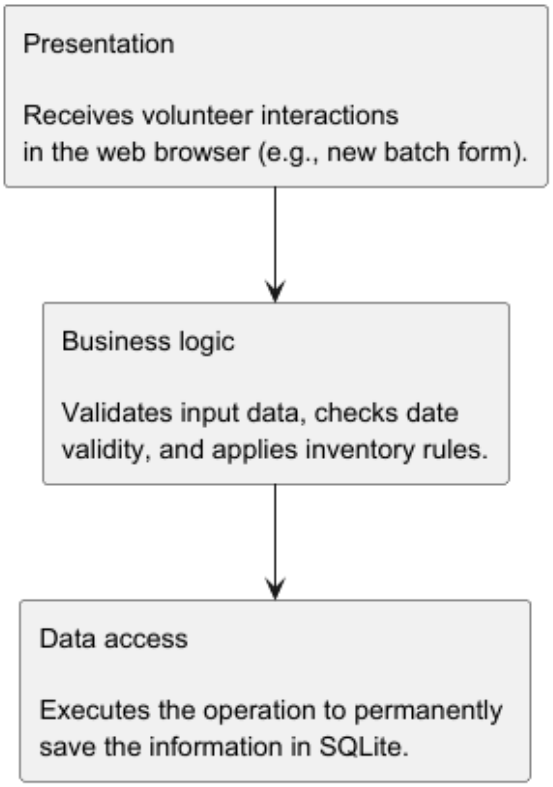


Figure 1: Component diagram.

3.4 Layer Decoupling Strategies

A strict rule in our architecture is that the frontend never interacts directly with the database. This prevents the design flaw known as Layer Bleeding.

To achieve this decoupling, StockDate uses two technical strategies:

1. **The Data Transfer Object (DTO) Pattern**

When the system queries the inventory, the Data Access layer returns a complete BatchEntity from SQLite. This entity contains internal data such as foreign keys (`category_id`, `user_id`) and audit fields (`createdAt`, `active`).

Sending this entity directly to the web interface would expose internal data and tie the frontend to the table structure. Instead, the service copies only what the screen needs (e.g., productName, quantity, daysUntilExpiration and status) into a lightweight object called a DTO. The presentation layer only receives DTOs. If the SQLite table structure changes later, the web interface keeps working because the DTO stays the same. The same

idea applies to incoming data: forms are sent as request DTOs (e.g., CreateBatchRequest), never as entities.

2. **Dependency Inversion via Interfaces**

The Business Logic layer does not depend on a specific SQLite connection class. Instead, it relies on Java interfaces (InventoryRepository, MovementRepository and UserRepository). Spring Boot injects the concrete implementation at runtime. If the team decides to replace SQLite with PostgreSQL later, we only need to change the database driver and the `application.properties` file. The business logic does not need any code changes because it only sees the interfaces.

3.5 Static View: UML Class Diagram

The complete class diagram is shown on the next page in landscape orientation so the attributes and methods can be read.

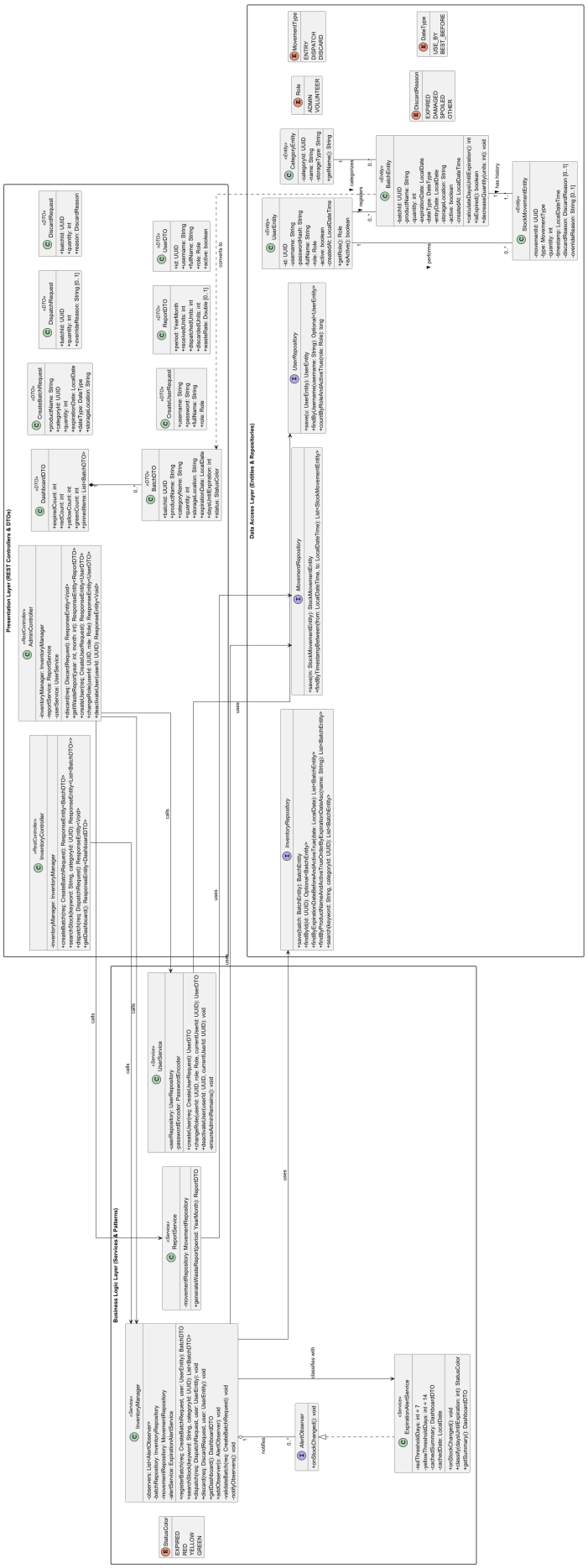


Figure 2: Complete UML Class Diagram.

3.6 Core Classes and Data Types

The domain logic of StockDate relies on four entities that map to SQLite tables. The class diagram uses visibility modifiers (+ public, - private) and explicit method signatures.

- **BatchEntity:** Represents a physical group of donated food. It uses UUID for batchId to prevent ID guessing in URLs. For dates (expirationDate, entryDate), it uses Java's LocalDate instead of Date to avoid time-zone shifting errors. The dateType field (USE_BY or BEST_BEFORE) tells the system whether an expired batch can still be dispatched. The method `+calculateDaysUntilExpiration(): int` contains the shelf-life logic, and the `active` flag marks batches that reached zero units.
- **StockMovementEntity:** Records every change in stock: ENTRY, DISPATCH or DISCARD, with quantity, timestamp, the user who made it and the discard or override reason when needed. These records are the source for the monthly report (UC-07) and the traceability that health auditors need.
- **UserEntity:** Stores the username, the BCrypt password hash, the full name, the `active` flag and the role. The role is an enum (ADMIN or VOLUNTEER), so invalid values cannot be stored.
- **CategoryEntity:** Groups products by storage requirements (like dry goods or refrigerated).

The diagram also defines enums for MovementType, DiscardReason, DateType and StatusColor (EXPIRED, RED, YELLOW, GREEN).

3.7 Interfaces and Data Transfer Objects (DTOs)

To keep the layers decoupled and avoid Layer Bleeding, StockDate separates input and output DTOs and hides the database behind interfaces.

- **Request DTOs (input):** CreateBatchRequest, DispatchRequest, DiscardRequest and CreateUserRequest contain exactly the fields of each form. For example, DispatchRequest has an optional overrideReason that is only filled when the user skips the FEFO batch.
- **Response DTOs (output):** BatchDTO exposes batchId, productName, categoryName, quantity, storageLocation, expirationDate, daysUntilExpiration and status. DashboardDTO groups the tier counts and the pinned items, ReportDTO holds the monthly totals and waste rate and UserDTO returns user data without the password hash.
- **Repository interfaces:** InventoryRepository defines operations such as listing all active batches in FEFO order (used by the dashboard) and searching by keyword and category (UC-03). MovementRepository returns the movements of a period for the

reports, and UserRepository supports login and the "last admin" check (`countByRole AndActiveTrue`).

3.8 Multiplicities and System Relationships

The structural relationships are defined with explicit multiplicities:

- **1:N (One-to-Many):** A UserEntity registers 0..* BatchEntity records, and each batch is linked to exactly one user for auditing.
- **1:N (One-to-Many):** A CategoryEntity categorizes 0..* batches, so items can be filtered by storage type.
- **1:N (One-to-Many):** A BatchEntity has 1..* StockMovementEntity records. The minimum is one because every batch is created together with its ENTRY movement.
- **1:N (One-to-Many):** A UserEntity performs 0..* movements, so every dispatch and discard can be traced to a person.

3.9 Applied Design Patterns

The dashboard alerts use the Observer Design Pattern.

- InventoryManager acts as the subject and holds a list of AlertObserver objects.
- Every operation that changes stock (registerBatch, dispatch and discard) calls `-notifyObservers()` after the transaction has been saved.
- ExpirationAlertService implements AlertObserver. When it is notified, it clears its cached dashboard summary. The next dashboard request rebuilds the summary from the database. The summary is also rebuilt when the server date changes, because days until expiration depend on the current date (UC-05, rule 1).
- The notification runs asynchronously (`@Async`, enabled with `@EnableAsync`), so the user's save request does not wait for it.

This means InventoryManager does not need to know how the dashboard works; it only announces that stock has changed. New observers could be added later (for example, a low-stock alert) without changing InventoryManager.

3.10 Dynamic View: Sequence Diagrams

3.10.1 Scenario 1: New Food Batch Registration Flow

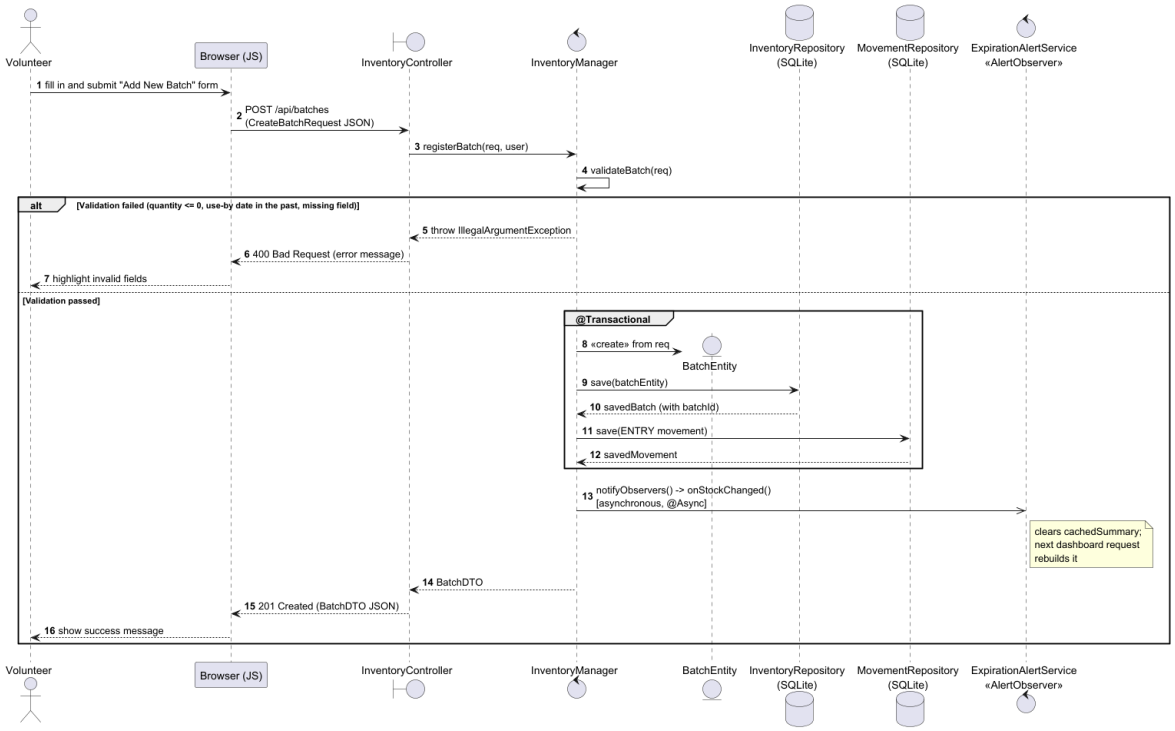


Figure 3: Sequence Diagram: Batch Registration Flow

Step-by-Step Execution Flow:

1. **User Interaction:** The volunteer fills out the web form with the details of a new food donation (product name, category, quantity, location, expiration date and date type) and clicks submit. The browser's JavaScript sends a POST request with a CreateBatchRequest in JSON to the InventoryController.

2. **Controller Delegation:** The InventoryController calls `registerBatch(req, user)` on the InventoryManager (Business Logic Layer), passing the logged-in user.

3. **Business Rule Validation:** Before touching the database, the InventoryManager validates the request: the quantity must be greater than zero, required fields must be present and a use-by date cannot be in the past.

4. **Error Scenario (Validation Failure):** If validation fails, the manager throws an IllegalArgumentException. The controller catches it and returns HTTP 400 Bad Request, and the frontend highlights the wrong fields. The database is never touched.

5. **Entity Creation and Persistence:** If validation passes, the manager creates a new BatchEntity and, inside a single transaction (`@Transactional`), saves it through InventoryRepository and saves the matching ENTRY movement through MovementRepository. Both records are saved or neither is (NFR-REL-02).

6. **Asynchronous Notification:** After the transaction, the manager calls `notifyObservers()`. The call to ExpirationAlertService is asynchronous, so the request continues without waiting. The service clears its cached dashboard summary.
7. **User Response:** The manager returns a BatchDTO, the controller sends HTTP 201 Created with it and the frontend shows a success message.

3.10.2 Scenario 2: Dashboard Alert Retrieval Flow

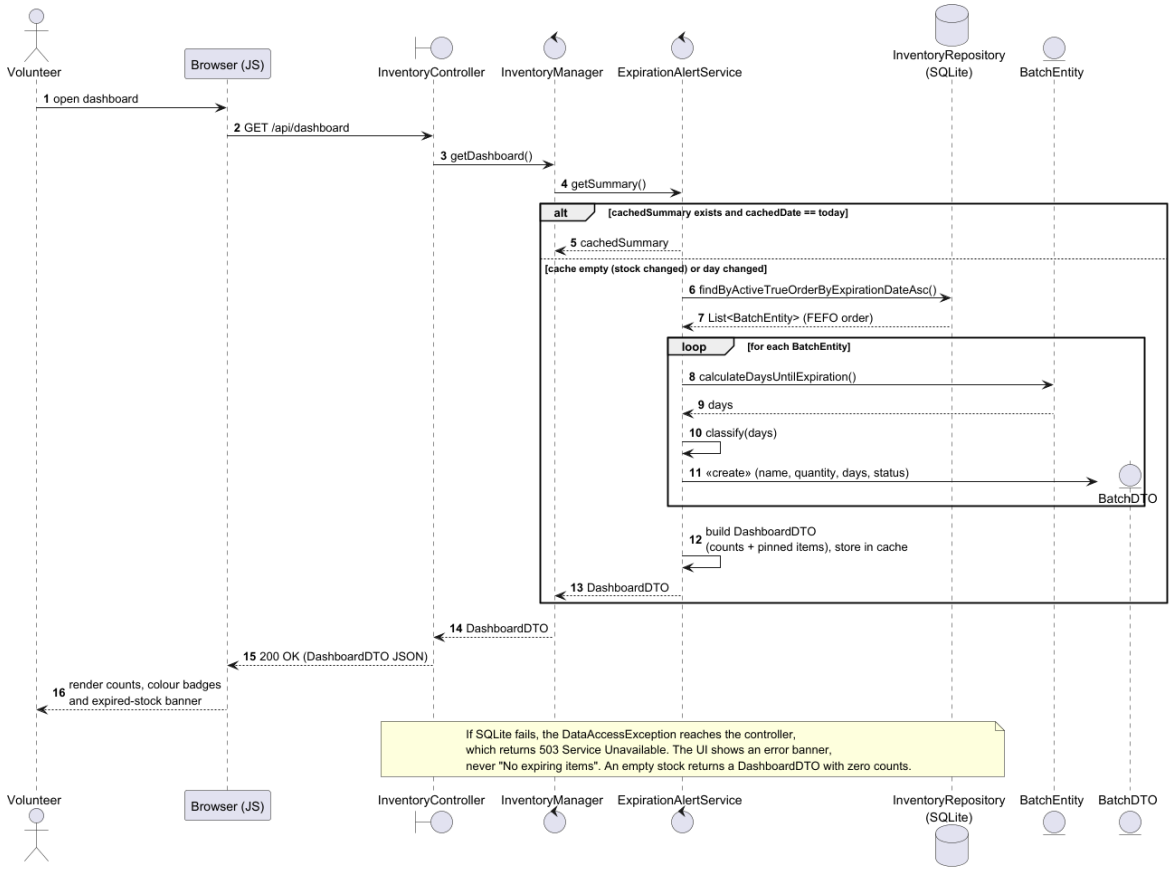


Figure 4: Sequence Diagram: Dashboard Alert Retrieval Flow

Step-by-Step Execution Flow:

1. **User Request:** The volunteer opens the dashboard. The browser's JavaScript sends GET `/api/dashboard` to the InventoryController.
2. **Service Invocation:** The controller calls `getDashboard()` on the InventoryManager, which asks ExpirationAlertService for the summary.
3. **Cache Check:** If a cached summary exists and was built today, it is returned directly.
4. **Rebuilding the Summary:** Otherwise, the service asks InventoryRepository for all active batches already sorted by expiration date (FEFO order).

5. **Classification and DTO Conversion (Loop):** For each batch, the service calls `calculateDaysUntilExpiration()`, classifies the result (Expired, Red, Yellow or Green) and creates a BatchDTO with only the data the view needs.
6. **Response:** The service builds a DashboardDTO with the count per tier and the pinned items (Expired and Red), stores it in the cache and returns it. The controller sends it as JSON with HTTP 200 OK.
7. **Rendering:** The JavaScript on the page draws the counts, the color badges with their text labels and, if there are expired batches, the top banner.

Error Scenarios: If SQLite fails, the DataAccessException reaches the controller, which returns HTTP 503 Service Unavailable, and the page shows an error banner. This case is never shown as "No expiring items", because a volunteer could then believe that nothing needs attention. If the stock is simply empty, the DashboardDTO has zero counts and the page says so.

3.11 Technology Decision Records (ADRs)

3.11.1 ADR-01: Selection of Backend Framework

1. **Status:** Accepted
2. **Context:** StockDate needs a server-side framework that can handle the expiration logic and provide an API for the web dashboard. The chosen technology must allow fast development (16-week deadline) and also meet the academic requirements for Phase 3, which require static code analysis with Java tools such as PMD and Checkstyle.
3. **Options Considered:**
   - **Option A: Java with Spring Boot**
     - **Query Performance:** Very good, with built-in support for multithreading and database connection pooling.
     - **Learning Curve:** Steep. The team has basic Java knowledge but no prior experience with the Spring ecosystem or its dependency injection model.
     - **Ecosystem:** Native compatibility with PMD, Checkstyle, JaCoCo and JUnit.
   - **Option B: Python with Django**
     - **Query Performance:** Good, but slower than compiled Java for heavy calculations.
     - **Learning Curve:** Low. The team already has experience with Django's MTV architecture.
     - **Ecosystem:** Excellent ORM, but PMD and Checkstyle do not support Python (alternatives like Pylint would need the instructor's approval).

4. **Decision & Outcome:** The team chose Option A: Java with Spring Boot (version 3.x, which requires Java 17 or newer).
5. **Consequences & Trade-offs:**
   - **What we gain:** Full compliance with the static analysis requirements (PMD/Checkstyle) of Phase 3 without asking for exceptions, and a typed backend that fits the Layered Architecture.
   - **What we sacrifice (Technical Debt):** A slower start. The team will spend the first week of coding on Spring Boot and REST API tutorials.

3.11.2 ADR-02: Selection of Database Engine

1. **Status:** Accepted
2. **Context:** StockDate needs a relational database for batches, movements, users and categories. The schema must be normalized (1NF, 2NF, 3NF) for academic compliance. At the same time, the target users are small food banks with no IT budget, no dedicated servers and no technical staff to maintain a database server.
3. **Options Considered:**
   - **Option A: SQLite (Embedded relational database).**
   - **Option B: PostgreSQL (Client-server relational database).**

Criteria	Option A: SQLite	Option B: PostgreSQL
Deployment Complexity	None. A single file next to the application.	High. Requires a separate service and port configuration.
Maintenance Cost	None.	Requires server hosting or local technical administration.
Query Performance	Fast for reads; locks the whole file on writes.	Very good for many concurrent reads and writes.
Normalization Support	Full relational constraints (Primary/Foreign keys).	Full relational constraints and stricter typing.
Spring Data JPA Support	Needs the community Hibernate dialect (`hibernate-community-dialects`).	Officially supported by Hibernate.

4. **Decision & Outcome:** The team chose Option A: SQLite as the primary database engine.

5. **Consequences & Trade-offs:** We gain a free database with no configuration, stored as a single file (`stockdate.db`) that is easy to back up and move. We give up smooth handling of many concurrent writes. We accept this because a local food bank will usually have only one or two volunteers registering batches at the same time. Two technical risks are accepted: SQLite needs the community Hibernate dialect, and UUID values must be stored as text, which has to be configured in the entity mapping.

3.11.3 ADR-03: Deployment Architecture

1. **Status:** Accepted
2. **Context:** The system needs a deployment strategy for daily testing and final delivery. The development cycle is limited to 16 weeks. The team members work on different operating systems but have no experience with containerization tools. Learning Docker from scratch would take time away from the core business logic and database work.
3. **Options Considered:**
   - **Option A: Direct Local Deployment.** Running the Spring Boot application from the IDE (IntelliJ) or as a compiled executable `.jar` file on the host machine.
   - **Option B: Docker Containerization.** Packaging the application and its embedded SQLite database into an isolated container.

Criteria	Option A: Direct Local Deployment	Option B: Docker Containerization
Team Learning Curve	None. The team already uses IntelliJ to run Java projects.	High. Requires learning Dockerfiles, port binding, and volumes.
Development Speed	Fast. Immediate feedback during coding.	Slow at first due to setup problems and image building.
Environment Consistency	Low. Prone to "it works on my machine" bugs between different operating systems.	Very high. The container behaves the same on any machine.

4. **Decision & Outcome:** The team chose Option A: Direct Local Deployment, delivering the application as a single executable JAR. Docker containerization is moved to an optional backlog task, to be done only if time allows at the end of the project.
5. **Consequences & Trade-offs:** We gain development time to focus on the backend and the frontend. We accept possible configuration differences between team members' computers; to reduce them, everyone uses the same Java version (17) and the Maven wrapper. The end user will need Java installed instead of running a single container

command. To keep this simple for non-technical staff, the delivery will include a short start script (`start.bat` / `start.sh`) and installation instructions.

3.11.4 ADR-04: Web Rendering Strategy

1. **Status:** Accepted
2. **Context:** We need to decide how the web dashboard will receive and display the inventory data. Four team members work at the same time, and the presentation layer must stay isolated from the business logic to prevent layer bleeding.
3. **Options Considered:**
   - **Option A: Server-Side Rendering (SSR) with Thymeleaf.** Spring Boot processes the HTML templates on the server, injects the Java objects into them, and sends the finished page to the browser.
   - **Option B: Decoupled REST API with Client-Side JavaScript.** The backend acts only as an API and returns JSON (DTOs). The frontend uses plain JavaScript to fetch this data and update the screen.

Criteria	**Option A: Thymeleaf (SSR)**	**Option B: REST API + Client JS**
Layer Decoupling	Low. HTML templates depend on the Java object structure.	High. The frontend only interacts with JSON endpoints.
Parallel Development	Difficult. UI and backend changes often touch the same files.	Easy. Team members can work on the UI and backend separately.
Learning Curve	Medium. Requires learning Thymeleaf syntax.	Low. Uses standard HTML, CSS, and JavaScript.

4. **Decision & Outcome:** The team chose Option B: Decoupled REST API with Client-Side JavaScript.
5. **Consequences & Trade-offs:** We gain a clear separation between frontend and backend. Two team members can build the screens using mock JSON while the other two write the services and queries, which reduces merge conflicts in GitHub. We accept writing more code by hand: instead of Spring Boot filling in the tables, the team must write JavaScript `fetch()` calls, handle loading and error states, and update the HTML manually.

3.12 Architectural Conclusions

The Layered Architecture implemented with Spring Boot keeps the web interface, the business rules and the SQLite database separate. DTOs and repository interfaces are what make this separation real: the frontend only sees JSON, and the services only see interfaces. This lets the team work on frontend and backend in parallel and test the core logic (FEFO, expiration status, waste rate) without a running database. The main debts we accept are the learning curve of Spring Boot (ADR-01), SQLite's single writer (ADR-02) and delivering a JAR instead of a container (ADR-03).

3.13 Requirement-to-Component Traceability Matrix

This matrix shows which architectural components implement each goal, use case and the main non-functional requirements.

Table 12: Requirement-to-component traceability

Requirement / Goal	Main Components	Responsibility
Goal 1: Fast Batch Registration / UC-02	InventoryController, CreateBatchRequest, InventoryManager	Short form with only the needed fields; validation and save in one request.
Goal 2: Expiration Alerts / UC-05	ExpirationAlertService (Observer), DashboardDTO	Classifies batches by days left and keeps the dashboard summary up to date.
Goal 3: Query Performance / UC-03	InventoryRepository, InventoryManager	Search queries filtered by keyword and category, sorted by expiration date.
Goal 4: Code Reliability / NFR-MAIN-01	InventoryManager, ExpirationAlertService, ReportService	All business rules live in services that can be unit tested with mocked repositories.
Goal 5: Ease of Use / NFR-USA-01	Presentation layer (HTML/CSS/JS)	Simple screens with no multi-level menus; status shown with color and text.
UC-01: Login	Spring Security, UserRepository, UserEntity	Checks the BCrypt hash and the active flag, and creates the session.

Requirement / Goal	**Main Components**	**Responsibility**
UC-04: Dispatch (FEFO)	InventoryManager, DispatchRequest, MovementRepository	Offers batches in FEFO order, blocks expired use-by batches, stores DISPATCH movements and override reasons.
UC-06: Discard	AdminController, InventoryManager, DiscardRequest	Admin-only write-off with mandatory reason; stores DISCARD movements.
UC-07: Waste Report	AdminController, ReportService, MovementRepository, ReportDTO	Adds up the movements of a month and calculates the waste rate.
UC-08: User Management	AdminController, UserService, UserRepository	Creates and edits users; prevents removing the last active admin.
NFR-SEC-02: Route Protection	AdminController (`/api/admin/**`), Spring Security	Rejects VOLUNTEER sessions with HTTP 403.
NFR-REL-02: Atomic Writes	InventoryManager (`@Transactional`)	Saves each batch change and its movement in the same transaction.
Stakeholder: Health Auditors	StockMovementEntity	Permanent record of every entry, dispatch and discard with user and timestamp.

3.14 Technical References

- Spring Boot Reference Documentation (3.x): `https://docs.spring.io/spring-boot/`. Layered applications, dependency injection, REST controllers, Actuator health checks and `@Async`.
- Spring Data JPA Reference Documentation: `https://docs.spring.io/spring-data/jpa/reference/`. Repository interfaces and derived query methods.
- Hibernate ORM community dialects: `https://github.com/hibernate/hibernate-orm/tree/main/hibernate-community-dialects`. SQLite dialect for Hibernate 6.
- SQLite Documentation: `https://www.sqlite.org/docs.html`. Transactions, locking and the integrity check.
- OMG Unified Modeling Language (UML) Specification 2.5.1: `https://www.omg.org/spec/UML/`. Notation for class diagrams (visibility, multiplicities) and sequence diagrams (asynchronous messages).
- W3C Web Content Accessibility Guidelines (WCAG) 2.1: `https://www.w3.org/TR/WCAG21/`. Contrast requirements used in NFR-USA-02.
- Architectural Decision Records (ADR GitHub organization): `https://adr.github.io/`. Templates used for the ADRs in this document.